Status	Finished
Started	Tuesday, 15 September 2026, 9:06 AM
Completed	Tuesday, 15 September 2026, 9:37 AM
Duration	31 mins 1 sec
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

You are choreographing a circus show with various animals. For one act, you are given two kangaroos on a number line ready to jump in the positive direction.
•The first kangaroo starts at position x1 and moves at a speed v1 meters per jump.
•The second kangaroo starts at position x2 and moves at a speed of v2 meters per jump and x2 > x1
•You have to figure out to get both kangaroos at the same position at the same time as part of the show before k jumps. If it is possible, return YES, otherwise return NO.

Input Format:
x1-position of kangaroo1
v1-Speed of kangaroo1
x2-position of kangaroo2
v2-Speed of kangaroo2
k-jumps

Output Format:
Both kangaroos are at the same position within k jumps, YES, otherwise NO.

For example:

Input	Result
0 3 4 2 6	YES

Answer: (penalty regime: 0 %)

```
import sys

def solve():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    x1, v1, x2, v2, k = map(int, input_data)

    if v1 <= v2:
        print("NO")
    elif (x2 - x1) % (v1 - v2) == 0:
        jumps = (x2 - x1) // (v1 - v2)
        if jumps <= k:
            print("YES")
        else:
            print("NO")
    else:
        print("NO")

if __name__ == '__main__':
    solve()

```

	Input	Expected	Got	
✓	0 3 4 2 6	YES	YES	✓
✓	0 3 2 4 8	NO	NO	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of the number of prime numbers in a specified range.

The starting and ending number of the range will be provided as input to the program.

Assumption: 2 <=starting number of the range<= ending number of the range<=7919

Example1: If the starting and ending number or the range is given as 2 and 20, the program must return 8, because there are 8 prime numbers in the specified range from 2 to 20. namely (2. 3. 5, 7, 11, 13, 17, 19)

Example2: If the starting and ending number of the range is given as 700 and 725, the program must return 3, because there are 3 prime numbers in the specified range from 700 to 725, namely (701, 709, 719)

For example:

Input	Result
2 20	8
700 725	3

Answer: (penalty regime: 0 %)

```
import sys

def count_primes_in_range():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    start_num = int(input_data[0])
    end_num = int(input_data[1])

    # Create a boolean array "is_prime[0..end_num]"
    is_prime = [True] * (end_num + 1)
    is_prime[0] = is_prime[1] = False

    p = 2
    while p * p <= end_num:
        if is_prime[p]:
            for i in range(p * p, end_num + 1, p):
                is_prime[i] = False
        p += 1

    # Count primes in range [start_num, end_num]
    prime_count = sum(1 for i in range(start_num, end_num + 1) if is_prime[i])
    print(prime_count)

if __name__ == '__main__':
    count_primes_in_range()
```

	Input	Expected	Got	
✓	2 20	8	8	✓
✓	700 725	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

An automorphic number is a number whose square ends with the number itself.

For example, 5 is an automorphic number because 5*5 =25. The last digit is 5 which same as

the given number.

If it is an automorphic number display “Automorphic” else display “Not Automorphic”.

Input Format:

Take a Integer from Keyboard

Output Format:

Print Automorphic if given number is Automorphic number, otherwise Not Automorphic

Example input:

5

Output:

Automorphic

Example input:

25

Output:

Automorphic

Example input:

7

Output:

Not Automorphic

Answer: (penalty regime: 0 %)

```
import sys

def check_automorphic():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    num_str = input_data[0]
    n = int(num_str)
    square_str = str(n * n)

    # Check if the square string ends with the original number string
    if square_str.endswith(num_str):
        print("Automorphic")
    else:
        print("Not Automorphic")

if __name__ == '__main__':
    check_automorphic()
```

	Input	Expected	Got	
✓	5	Automorphic	Automorphic	✓
✓	625	Automorphic	Automorphic	✓
✓	7	Not Automorphic	Not Automorphic	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a number N, find the next perfect square greater than N.

Input Format:

Integer input from stdin.

Output Format:

Perfect square greater than N.

Example Input:

10

Output:

16

For example:

Input	Result
10	16
20	25

Answer: (penalty regime: 0 %)

```
import sys
import math

def main():
    input_data = sys.stdin.read().split()
    for item in input_data:
        n = int(item)
        root = math.isqrt(n)
        next_square = (root + 1) ** 2
        print(next_square)

if __name__ == '__main__':
    main()
```

	Input	Expected	Got	
✓	10	16	16	✓
✓	20	25	25	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write python program to print the following pattern based on input size.

Input:

3

Output:

```
  1
 2 3
4 5 6
```

For example:

Input	Result
4	1 2 3 4 5 6 7 8 9 10

Answer: (penalty regime: 0 %)

```
import sys

def print_pattern():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    current_num = 1

    for i in range(1, n + 1):
        # Leading spaces for alignment
        print(" " * (n - i), end="")

        # Numbers in the current row
        row_numbers = []
        for _ in range(i):
            row_numbers.append(str(current_num))
            current_num += 1

        print(" ".join(row_numbers))

if __name__ == '__main__':
    print_pattern()
```

	Input	Expected	Got	
✓	3	1 2 3 4 5 6	1 2 3 4 5 6	✓
✓	4	1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

A Number is said to be Disarium number when the sum of its digit raised to the power of their respective positions becomes equal to the number itself. Write a program to print number is Disarium or not.

Input Format:

Single Integer Input from stdin.

Output Format:

Yes or No.

Example Input:

175

Output:

Yes

Explanation

1^1 + 7^2 +5^3 = 175

Example Input:

123

Output:

No

For example:

Input	Result
175	Yes
123	No

Answer: (penalty regime: 0 %)

```
import sys

def is_disarium():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    num_str = input_data[0]
    n = int(num_str)

    # Calculate sum of digits raised to their 1-based position power
    total_sum = sum(int(digit) ** (index + 1) for index, digit in enumerate(num_str))

    if total_sum == n:
        print("Yes")
    else:
        print("No")

if __name__ == '__main__':
    is_disarium()
```

	Input	Expected	Got	
✓	175	Yes	Yes	✓
✓	123	No	No	✓
✓	89	Yes	Yes	✓
✓	90	No	No	✓

	Input	Expected	Got	
✓	518	Yes	Yes	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a positive integer a, find the smallest positive integer b whose multiplication of each digit equals to a.

If there is no answer or the answer is not fit in 32-bit signed integer, then return 0.

Example 1

Input:

48

Output:

68

Example 2

Input:

15

Output:

35

For example:

Input	Result
48	68
15	35

Answer: (penalty regime: 0 %)

```
import sys

def smallest_factor_digits(a):
    # Edge case: single-digit numbers directly yield themselves
    if a < 10:
        return a

    digits = []

    # Greedily extract single-digit factors from 9 down to 2
    for factor in range(9, 1, -1):
        while a % factor == 0:
            digits.append(str(factor))
            a //= factor

    # If a is not reduced to 1, prime factors > 9 exist
    if a > 1:
        return 0

    # Sort digits in ascending order to form the smallest integer
    result_str = "".join(reversed(digits))
    result = int(result str)
```

```
23
24      # Check 32-bit signed integer upper limit (2^31 - 1 = 2147483647)
25      if result > 2147483647:
26          return 0
27
28      return result
29
30  if __name__ == '__main__':
31      input_data = sys.stdin.read().split()
32      if input_data:
33          n = int(input_data[0])
34          print(smallest_factor_digits(n))
```

	Input	Expected	Got	
✓	48	68	68	✓
✓	15	35	35	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Write a program to find the count of non-repeated digits in a given number N. The number will be passed to the program as an input of type int.

Assumption: The input number will be a positive integer number >= 1 and <= 25000.

Some examples are as below.

If the given number is 292, the program should return 1 because there is only 1 non-repeated digit '9' in this number

If the given number is 1015, the program should return 2 because there are 2 non-repeated digits in this number, '0', and '5'.

If the given number is 108, the program should return 3 because there are 3 non-repeated digits in this number, '1', '0', and '8'.

If the given number is 22, the function should return 0 because there are NO non-repeated digits in this number.

For example:

Input	Result
292	1
1015	2
108	3
22	0

Answer: (penalty regime: 0 %)

```
import sys
from collections import Counter

def count_non_repeated_digits():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    num_str = input_data[0]

    # Count frequency of each digit character in the input string
    digit_counts = Counter(num_str)

    # Count how many digits have a frequency of exactly 1
    non_repeated_count = sum(1 for count in digit_counts.values() if count == 1)

    print(non_repeated_count)

if __name__ == '__main__':
    count_non_repeated_digits()
```

	Input	Expected	Got	
✓	292	1	1	✓
✓	1015	2	2	✓
✓	108	3	3	✓
✓	22	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Let's print a chessboard!

Write a program that takes input:

Integer N(represents the rows and columns of a chessboard) and also the starting character of the chessboard

Output Format

Print the chessboard as per the given examples

Sample Input / Output

Input:

2

W

Output:

WB

BW

Answer: (penalty regime: 0 %)

```
import sys

def print_chessboard():
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    n = int(input_data[0])
    start_char = input_data[1].upper()

    # Determine character mapping based on starting character
    char1 = start_char
    char2 = 'B' if start_char == 'W' else 'W'

    for row in range(n):
        row_str = []
        for col in range(n):
            # Alternating characters based on row and column parity
            if (row + col) % 2 == 0:
                row_str.append(char1)
            else:
                row_str.append(char2)
        print("".join(row_str))

if __name__ == '__main__':
    print_chessboard()
```

	Input	Expected	Got	
✓	2 W	WB BW	WB BW	✓
✓	3 B	BWB WBW BWB	BWB WBW BWB	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a program that given an integer ‘n’, prints the number of integers that are less than or equal to ‘n’ and co-prime to ‘n’

Two integers a and b are said to be relatively prime or co-prime if the only positive integer that evenly divides both of them is 1. That is, the only common positive factor of the two numbers is 1. This is equivalent to their greatest common divisor being 1.

Input Format:

One line containing the value of 'n', where 1<=n<=10,000

Output Format:

One line containing the number of integers that are co-prime to n and less than or equal to 'n'

Sample Test Cases

Test Case 1

Input

10

Output

4

Test Case 2

Input

23

Output

22

Test Case 3

Input

11

Output

10

Answer: (penalty regime: 0 %)

```
import sys
import math

def euler_totient(n):
    result = n
    p = 2
    temp = n

    # Check prime factors up to sqrt(n)
    while p * p <= temp:
        if temp % p == 0:
            while temp % p == 0:
                temp //= p
            result -= result // p
        p += 1

    if temp > 1:
        result -= result // temp
```

```
        result = result // temp

    return result

if __name__ == '__main__':
    input_data = sys.stdin.read().split()
    if input_data:
        n = int(input_data[0])
        print(euler_totient(n))
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	23	22	22	✓
✓	11	10	10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.